European Commission

DPP Registry Service Account Technical Integration Guide for Economic Operators

DG Grow Digital Solutions

Last published 2026-10-30 v1.0



Table of Contents

Chapter 1. Introduction 4
1.1 About this Guide 4
Target Audience 4
Scope and Coverage 4

Chapter 2. Service Account Integration Overview 5
2.1 How Service Account Authentication Works 5
2.2 Main Identifiers and Concepts 5

Chapter 3. Getting started 6

Chapter 4. Key Pair and Credential Requirements 7
4.1 Supported Cryptographic Profiles 7
4.2 Generate and Protect the Key Pair 7
Example with OpenSSL 7
4.3 Public-Key Formats Accepted by the Registry 8
4.4 Credential kid and Thumbprint 8
4.5 Credential Validity 8

Chapter 5. Obtaining a DPP Access Token 9
5.1 Authorization Server Endpoints 9
5.2 Build the Client Assertion 9
Protected header 9
Claims 10
5.3 Sign the Client Assertion 10
5.4 Send the Token Request 10
5.5 Token Response 11

Chapter 6. Using the Access Token with Registry APIs 12
6.1 Resource Identifier and Audience 12
6.2 Example Registry Request 12

Chapter 7. Complete Integration Examples 13
7.1 Minimal Configuration 13
7.2 Java - Create a Client Assertion 13
7.3 TypeScript / Node.js - Create a Client Assertion 13

Chapter 8. Token Lifecycle and Recommended Client Behaviour 15

Chapter 9. Credential Lifecycle 16
9.1 Adding a Second Credential 16
9.2 Switching to a New Credential 16
9.3 Revoked and Expired Credentials 16
9.4 Not operational Service Account 16

Chapter 10. Service Account Deletion and Access Impact 17

Chapter 11. Error Handling and Troubleshooting 18
11.1 Troubleshooting checklist for invalid_client 18

Chapter 12. Security Recommendations for EO Integrations 19

Chapter 13. DPP Registry Environments and Endpoints 20

Chapter 14. Integration Validation Checklist 21

Chapter 15. API and Authentication Reference 22

Chapter 16. Glossary of Terms 23

Chapter 17. Helpdesk Contacts 24

NOTE

This guide describes the integration that an Economic Operator implements in its own systems after a Service Account and at least one credential has been created in the DPP Registry.

Chapter 1. Introduction

1.1 About this Guide

This guide explains how an Economic Operator (EO) can integrate an external system with the DPP Registry using a Service Account. Follow this guide after the Service Account has been created and a public-key credential has been registered.

Target Audience

This guide is intended for EO developers, system integrators, DevOps engineers and security teams responsible for automated DPP Registry integrations.

Scope and Coverage

This document provides relevant information and instructions regarding:

- Service Account integration prerequisites and identifiers.
- Supported EC key profiles and credential handling.
- OAuth 2.0 Client Credentials authentication using **private_key_jwt**.
- DPP access-token acquisition and reuse.
- Calling DPP Registry APIs with Bearer authentication.
- Credential lifecycle, troubleshooting and security recommendations.

Chapter 2. Service Account Integration Overview

A Service Account is a non-human identity bound to a verified Economic Operator. It enables an external EO system to authenticate without a browser session or EU Login interaction.

2.1 How Service Account Authentication Works

Step	Action	Result
1	Create Service Account	An EO Administrator creates a Service Account and registers a public credential in the DPP Registry.
2	Create client assertion	The EO system creates a short-lived JWT and signs it with the corresponding private EC key.
3	Obtain access token	The EO system sends the signed assertion to the token endpoint using OAuth 2.0 Client Credentials.
4	Call Registry API	The Authorization Server returns a short-lived Bearer access token that the EO system uses to call DPP Registry APIs.

2.2 Main Identifiers and Concepts

Item	Used for
Service Account Alias	Human-readable name of the Service Account.
Credential ID	Identifier of one registered public-key credential.
Key ID (kid)	Key identifier included in the client-assertion protected header. It identifies the credential used to verify the assertion.
Private key	Held only by the EO and used to sign client assertions. Never upload or send it anywhere.
Public key	Registered as a Service Account credential and used by the Authorization Server to verify client assertions.

TIP
Store the Client ID and kid in your application configuration. Store the private key in a protected key store, HSM, KMS or secrets-management solution.

Chapter 3. Getting started

Before you implement or test the machine-to-machine authentication flow, verify that the Service Account and all required integration information are ready:

- Your Organisation is **VERIFIED** in the DPP Registry.
- An EO Administrator has created the Service Account. A maximum of 5 Operational Service Accounts is allowed per Organisation.
- At least one **ACTIVE** credential is associated with the Service Account.
- You have the **Client ID** issued by the registry.
- You have the **kid** of the credential that your system will use.
- You have secure access to the private key matching the registered public key.
- You know the algorithm/curve used for each Service Account credential.
- You know the token endpoint of the **Authentication Service** acting as the OAuth 2.0 Authorization Server, the target resource identifier and the Registry API base URL for the target environment.

> **NOTE**
> The Service Account's Client ID is public configuration data. The private key is not. Never copy the private key into application source code, tickets, email, chat, logs or Registry support requests.

Chapter 4. Key Pair and Credential Requirements

4.1 Supported Cryptographic Profiles

Curve	JWT algorithm	Supported
P-256	ES256	Yes
P-384	ES384	Yes
P-521	ES512	Yes

4.2 Generate and Protect the Key Pair

Whenever possible, generate and store your private key in a **Hardware Security Module (HSM)**.

- **Step 1: Generate the private key inside the HSM.**

  Configure the private key as non-exportable and use an HSM that generates keys internally using a cryptographically secure random-number generator (CSPRNG).

- **Step 2: Export only the corresponding public key.**

  Register only the public key with the DPP Registry. Never export or provide the private key.

> **TIP**
> Use an HSM that generates the key internally using a cryptographically secure random-number generator (CSPRNG) and prevents export of the private key.

Example with OpenSSL

Where the use of a Hardware Security Module (HSM) is not feasible, you may generate cryptographic key pairs using software-based tools, such as OpenSSL, at its sole risk and responsibility.

The following example generates an encrypted P-256 private key and derives the corresponding public key for upload to the DPP Registry.

1. Generate the encrypted private key

```
openssl genpkey \
  -algorithm EC \
  -pkeyopt ec_paramgen_curve:P-256 \
  -aes-256-cbc \
  -out private-key.pem
```

OpenSSL prompts for a passphrase. The -aes-256-cbc option encrypts the private-key file at rest. Store the passphrase in secure secret storage, separately from the private key, and do not hard-code it in application source or scripts.

Run key generation only on an operating system providing a cryptographically secure random-number generator (CSPRNG) and using a maintained OpenSSL version.

2. Derive the public key

```
openssl pkey \
  -in private-key.pem \
  -pubout \
  -out public.pem
```

3. Upload only the public key.
   Upload only **public.pem** to the Registry. Never upload or otherwise provide private-key.pem to the Registry.

> **SECURITY**
> The Economic Operator is responsible for generating, protecting and controlling the authorised use of the private keys corresponding to credentials registered in the registry. The registry stores only the public credential and cannot recover the EO private key.

4.3 Public-Key Formats Accepted by the Registry

Format	Typical extension	Notes
JWK JSON	.json	Public EC JWK only.
PEM public key	.pem	The OpenSSL example above produces a suitable public PEM key.

The Registry rejects private-key material in every supported input representation.

4.4 Credential kid and Thumbprint

Each credential has a key ID (kid). Include the kid in the protected header of every client assertion signed with that credential.

The registry also calculates an RFC 7638 SHA-256 JWK thumbprint from the normalized public key. The thumbprint identifies the public key independently of its kid. Changing the kid does not allow a previously registered public key to be registered again.

4.5 Credential Validity

A credential becomes valid immediately when it is registered. The **recommended credential validity period is one year**. The EO Administrator may select a longer validity period when needed, up to a **maximum of two years**.

A Service Account may have a **maximum of 2 ACTIVE** credentials at the same time, allowing you to switch credentials without interrupting access.

Chapter 5. Obtaining a DPP Access Token

Your system authenticates using OAuth 2.0 Client Credentials with **private_key_jwt**. Shared client secrets are not supported.

5.1 Authorization Server Endpoints

The following values apply to the Acceptance and Production environments. Refer to **Chapter 13** for the complete environment endpoint reference.

Parameter	Acceptance	Production
Authorization Server issuer	https://auth.acc.product-passport.ec.europa.eu	https://auth.product-passport.ec.europa.eu
Token endpoint	https://auth.acc.product-passport.ec.europa.eu/oauth2/token	https://auth.product-passport.ec.europa.eu/oauth2/token
Authorization Server metadata	https://auth.acc.product-passport.ec.europa.eu/.well-known/oauth-authorization-server	https://auth.product-passport.ec.europa.eu/.well-known/oauth-authorization-server
DPP access-token JWKS	https://auth.acc.product-passport.ec.europa.eu/oauth2/jwks	https://auth.product-passport.ec.europa.eu/oauth2/jwks
Registry resource indicator	https://auth.acc.product-passport.ec.europa.eu/resources/dpp/registry	https://auth.product-passport.ec.europa.eu/resources/dpp/registry

> **NOTE**
> Authorization Server URLs and EU Login authentication URLs belong to separate endpoint families. Service Account implementations must use only the Authorization Server URLs documented for the corresponding environment.

5.2 Build the Client Assertion

For each token request, create a new short-lived signed JWT. Do not reuse a previous client assertion.

Protected header

```
{
  "typ": "client-authentication+jwt",
  "alg": "ES256",
  "kid": "key-2026-09"
}
```

Use the algorithm bound to your credential: ES256, ES384 or ES512.

Claims

```
{
  "iss": "<clientId>",
  "sub": "<clientId>",
  "aud": "https://auth.product-passport.ec.europa.eu",
  "iat": <current Unix time>,
  "exp": <iat + 60 to 120 seconds>,
  "jti": "<new unique value>"
}
```

Claim	Requirement
iss	Exactly the registry Client ID.
sub	Exactly the registry Client ID.
aud	Exact Authorization Server issuer for the environment.
iat	Current issue time. Keep the your system clock synchronized.
exp	Set exp 60–120 seconds after iat. The absolute maximum is 5 minutes.
jti	Generate a fresh unique identifier for every assertion. Never reuse a jti.

5.3 Sign the Client Assertion

Sign the JWT with the EC private key corresponding to the active credential identified by kid. Use a JOSE/JWT library that supports the selected ECDSA algorithm. Do not implement JWT ECDSA signature encoding manually.

> **TIP**
> Keep the private key within the EO environment. Send only the signed client assertion. Never send the private key.

5.4 Send the Token Request

Send the token request to the Authorization Server using application/x-www-form-urlencoded.

Example token request

```
curl -X POST '  <Authorization Server token endpoint>' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'grant_type=client_credentials' \
  --data-urlencode 'client_id=<clientId>' \
  --data-urlencode 'client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer'
\
  --data-urlencode 'client_assertion=<signed-client-assertion>' \
  --data-urlencode 'resource=<registry-resource-identifier>' \
  --data-urlencode 'scope=<resource-specific-scope>'
```

Parameter	Required	Value
grant_type	Yes	client_credentials
client_id	Yes	The registry-generated clientId of the Service Account.
client_assertion_type	Yes	urn:ietf:params:oauth:client-assertion-type:jwt-bearer
client_assertion	Yes	The signed private_key_jwt client assertion.

Parameter	Required	Value
resource	Yes	HTTPS URI identifying the target Registry Resource Server for the selected environment.
scope	No	Requested access within the target Resource Server, when applicable.

When **scope** is provided, it can only request access already permitted for the Service Account. It cannot increase the Service Account's authorization.

5.5 Token Response

```
{
  "access_token": "<signed DPP access token>",
  "token_type": "Bearer",
  "expires_in": 900,
  "scope": "<granted-resource-specific-scope>"
}
```

The access token is a Bearer JWT with a **maximum lifetime of 15 minutes**.

> **SECURITY**
> Rate limits apply to the authentication flow. Securely retain and reuse the issued access token until shortly before it expires. Do not request a new token for every API call. When a new token is required, authenticate again using a fresh client assertion.

Chapter 6. Using the Access Token with Registry APIs

Send the access token in the HTTP Authorization header when calling an API covered by the token.

Bearer authentication

```
Authorization: Bearer <access_token>
```

6.1 Resource Identifier and Audience

The resource parameter identifies the Resource Server for which the access token is requested.

For DPP Registry access, use the Registry resource identifier corresponding to the target environment:

- **Acceptance:**
  https://auth.acc.product-passport.ec.europa.eu/resources/dpp/registry
- **Production:**
  https://auth.product-passport.ec.europa.eu/resources/dpp/registry

These values are logical OAuth resource identifiers managed under the corresponding DPP Authorization Server domain. They are not Registry API endpoints and must not be replaced with the Registry API base URL.

The Authorization Server binds the issued access token to the requested resource through its audience **(aud)**. Use the access token only with Registry APIs configured to accept that exact audience.

> **NOTE**
> The resource identifier is
> onment-specific. Always use the value corresponding to the Authorization Server and Registry environment being accessed.

6.2 Example Registry Request

Submit a DPP registration request

```
curl -X POST ' https://reg.product-passport.ec.europa.eu/api/v1/dpp/submission' \
  -H 'Authorization: Bearer <access_token>' \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: <unique-request-id>' \
  -d '< registration request payload>'
```

Before calling the endpoint, ensure that the access token has the required audience and effective permissions. Registry API request payloads and business rules are documented separately in the Registry API documentation.

Chapter 7. Complete Integration Examples

7.1 Minimal Configuration

Configure at least the following values for each environment:

```
DPP_ISSUER=https://auth.product-passport.ec.europa.eu
DPP_TOKEN_ENDPOINT=https://auth.product-passport.ec.europa.eu/oauth2/token
DPP_RESOURCE=https://auth.product-passport.ec.europa.eu/resources/dpp/registry
DPP_REGISTRY_BASE_URL=https://reg.product-passport.ec.europa.eu/api/v1
DPP_CLIENT_ID=<issued by the registry>
DPP_KID=<active credential kid>
DPP_ALG=ES256
```

7.2 Java - Create a Client Assertion

The example below uses Nimbus JOSE + JWT. The example assumes that the EC private key has already been loaded securely from your keystore/HSM/KMS. Do not hard-code private keys or passphrases.

```
import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.ECDSASigner;
import com.nimbusds.jwt.*;
import java.security.interfaces.ECPrivateKey;
import java.time.Instant;
import java.util.*;

String clientId = System.getenv("DPP_CLIENT_ID");
String kid = System.getenv("DPP_KID");
String issuer = "https://auth.product-passport.ec.europa.eu";
ECPrivateKey privateKey = loadPrivateKeyFromSecureStore();

Date now = Date.from(Instant.now());
Date exp = Date.from(Instant.now().plusSeconds(120));

JWTClaimsSet claims = new JWTClaimsSet.Builder()
    .issuer(clientId)
    .subject(clientId)
    .audience(issuer)
    .issueTime(now)
    .expirationTime(exp)
    .jwtID(UUID.randomUUID().toString())
    .build();

JWSHeader header = new JWSHeader.Builder(JWSAlgorithm.ES256)
    .type(new JOSEObjectType("client-authentication+jwt"))
    .keyID(kid)
    .build();

SignedJWT jwt = new SignedJWT(header, claims);
jwt.sign(new ECDSASigner(privateKey));
String clientAssertion = jwt.serialize();
```

For a P-384 or P-521 Service Account, use ES384 or ES512 respectively.

7.3 TypeScript / Node.js - Create a Client Assertion

The example uses the jose library and assumes that privateKey has been loaded from a protected key store or secret-management integration.

```
import { SignJWT } from 'jose';
import { randomUUID } from 'node:crypto';

const clientId = process.env.DPP_CLIENT_ID!;
const kid = process.env.DPP_KID!;
const issuer = 'https://auth.product-passport.ec.europa.eu';
const now = Math.floor(Date.now() / 1000);

const clientAssertion = await new SignJWT({})
  .setProtectedHeader({
    typ: 'client-authentication+jwt',
    alg: 'ES256',
    kid
  })
  .setIssuer(clientId)
  .setSubject(clientId)
  .setAudience(issuer)
  .setIssuedAt(now)
  .setExpirationTime(now + 120)
  .setJti(randomUUID())
  .sign(privateKey);
```

After creating the assertion, use the token request described in Section 5.4.

Chapter 8. Token Lifecycle and Recommended Client Behaviour

- Access tokens have a **maximum lifetime of 15 minutes**.
- Reuse a valid access token until shortly before it expires. Do not request a new token for every API call. The Authentication Service applies rate limits.
- To obtain another access token, generate a fresh client assertion and call /oauth2/token again.
- Generate a new **jti** for every client assertion and never replay an assertion.
- Keep your system clock synchronized. Incorrect clocks may cause **iat** or **exp** validation failures.
- When the token endpoint returns 429 or a temporary availability error, apply bounded exponential backoff with jitter and respect Retry-After when provided.

Chapter 9. Credential Lifecycle

9.1 Adding a Second Credential

A Service Account can have **up to 2 ACTIVE credentials**. Use a second credential when changing the key used by an integration without interrupting access.

9.2 Switching to a New Credential

To change credentials, register the new credential first and revoke the old one only after the new credential is working.

- **Step 1:** Generate a new key pair and keep the new private key in the EO secure key store.
- **Step 2:** Register the new public key as a second credential in the DPP Registry.
- **Step 3:** Update your system to sign assertions with the new private key and new kid.
- **Step 4:** Confirm that your system can obtain and use a DPP access token with the new credential.
- **Step 5:** Revoke the old credential when it is no longer needed.

9.3 Revoked and Expired Credentials

A **revoked or expired credential** cannot be used to obtain new access tokens. Previously used public keys cannot be registered again, even with another kid.

9.4 Not operational Service Account

If all credentials are revoked or expired, the DPP Registry UI may display the Service Account as **Not Operational**. The practical effect is that the Service Account cannot obtain a new token until an ACTIVE credential is registered.

Chapter 10. Service Account Deletion and Access Impact

Deleting a Service Account is irreversible. **The Service Account is retained with status DELETED, and its associated credentials are no longer valid for authentication.** It cannot be reactivated, and its associated credentials can no longer be used to obtain new DPP access tokens.

> **NOTE**
> An access token issued before deletion is a self-contained Bearer token and may remain valid until its exp claim. The maximum residual lifetime is 15 minutes.

If you suspect that a credential has been compromised and immediate containment is required, revoke the affected credential or delete the Service Account. Treat any already-issued Bearer token as potentially usable until it expires.

Chapter 11. Error Handling and Troubleshooting

The token endpoint deliberately limits the detail returned for client-authentication failures. Multiple failure conditions may therefore be returned as invalid_client.

Response / error	Recommended check
invalid_request	Required form parameters, client_assertion_type, request encoding and one resource value.
invalid_client	Client ID; kid; registered algorithm/curve; assertion signature; iss/sub; aud; iat/exp; unique jti; credential status; Service Account and Organisation eligibility.
invalid_scope	Requested scope is not available to the Service Account or is not valid for the requested resource.
invalid_target	The requested resource is not supported or does not match the Registry resource identifier configured for the target environment.
temporarily_unavailable / 503	A required registry dependency is temporarily unavailable. Retry with bounded backoff and a fresh assertion.
429 Too Many Requests	Respect Retry-After when supplied and reduce token-request frequency. Reuse valid access tokens.
401/403 from Registry API	Confirm the access token is valid, has the Registry audience and contains the permissions required by the endpoint.

11.1 Troubleshooting checklist for invalid_client

- Confirm that request client_id exactly matches assertion iss and sub.
- Confirm that protected-header typ is client-authentication+jwt.
- Protected header kid identifies an ACTIVE credential.
- alg matches the curve/profile registered for the Service Account credentials.
- aud is exactly the environment Authorization Server issuer.
- iat and exp are valid and the system clock is synchronized.
- exp is no more than 5 minutes after iat. 60-120 seconds is recommended.
- jti is newly generated and has not been used before.
- The assertion was signed using the private key matching the registered public key.

Chapter 12. Security Recommendations for EO Integrations

- **Generate and store private keys in an HSM whenever possible.** The private key should be generated inside the HSM and configured as non-exportable.
- If an HSM cannot be used, generate the key on a maintained system backed by an operating-system **CSPRNG**, using an approved cryptographic tool such as OpenSSL.
- If a private key is stored in a file, encrypt it at rest and protect the encryption passphrase separately.
- Never send a private key anywhere. Only public-key material is registered.
- Never log private keys, client assertions or access tokens.
- Grant the Service Account only the capability needed for its business purpose.
- Use separate credentials for separate environments where possible.
- Monitor failed token requests, unexpected key usage and upcoming credential-expiration dates.

SECURITY

The recommended approach is to use an HSM so that the private key is generated and used without being exported from the secure hardware boundary. Use software-generated private keys only when an HSM is not available. Their protection and use remain the Economic Operator's responsibility.

Chapter 13. DPP Registry Environments and Endpoints

Use only the endpoint values published for your target environment. The following table lists the DPP Registry and Authorization Server endpoints for each environment.

Environment	Portal	Authorization Server Issuer	Token Endpoint	Registry API Base
Acceptance	https://registry.acc.product-passport.ec.europa.eu/	https://auth.acc.product-passport.ec.europa.eu	https://auth.acc.product-passport.ec.europa.eu/oauth2/token	https://reg.acc.product-passport.ec.europa.eu/api/v1
Production	https://registry.product-passport.ec.europa.eu/	https://auth.product-passport.ec.europa.eu	https://auth.product-passport.ec.europa.eu/oauth2/token	https://reg.product-passport.ec.europa.eu/api/v1

TIP

When available, retrieve Authorization Server metadata from /.well-known/oauth-authorization-server and use the issuer and token endpoint published there for the target environment.

Chapter 14. Integration Validation Checklist

- Confirm that the EO can access the correct DPP Registry environment and that the Organisation is VERIFIED.
- Confirm that the Service Account exists and that your system has the correct Client ID.
- Your system uses an ACTIVE credential and the correct kid.
- The private key is protected and matches the public key registered in the Registry.
- A fresh private_key_jwt can be generated with the correct typ, alg, iss, sub, aud, iat, exp and jti.
- The token request uses the Registry resource identifier for the target environment and succeeds with the required scope.
- Reuse the returned access token until shortly before expiration.
- Your system can call the intended Registry endpoint with Authorization: Bearer <access_token>.
- Generate a new client assertion for every token request.
- Ensure retry handling honours Retry-After and applies bounded backoff.
- Logs do not contain private keys, client assertions or access tokens.

Chapter 15. API and Authentication Reference

The following table provides the main Production references used for Service Account authentication and DPP Registry API integration. **Refer to Chapter 13** for the corresponding endpoints in other environments.

Reference	Current value / purpose
Authorization Server metadata	https://auth.product-passport.ec.europa.eu/.well-known/oauth-authorization-server
Token endpoint	https://auth.product-passport.ec.europa.eu/oauth2/token
DPP JWKS	https://auth.product-passport.ec.europa.eu/oauth2/jwks - DPP access-token verification keys.
Registry resource indicator	https://auth.product-passport.ec.europa.eu/resources/dpp/registry — logical resource identifier sent in the OAuth resource parameter when requesting a Registry access token. The issued access token uses this resource as its Registry audience.
Registry API base path	https://reg.product-passport.ec.europa.eu/api/v1
Registry portal	https://registry.product-passport.ec.europa.eu/

Refer to the Registry API contract for individual operations, payloads and endpoint-specific permissions. This guide covers the authentication and Service Account integration required to call those APIs.

Chapter 16. Glossary of Terms

Term	Meaning
Access token	Short-lived DPP Bearer JWT used to call a Registry API.
Client assertion	Short-lived JWT created and signed by the EO to authenticate its Service Account at the Authorization Server token endpoint.
Client ID	OAuth identifier generated for a Service Account.
Credential	Registered EO public key and associated metadata used to verify client assertions.
CSPRNG	Cryptographically secure random-number generator used as the entropy source when generating cryptographic keys.
Economic Operator (EO)	The organisation integrating its systems with the DPP Registry.
JWK	JSON Web Key representation of cryptographic key material.
Key ID (kid)	Key identifier included in the client assertion header to select the registered credential.
private_key_jwt	OAuth client-authentication method in which the client proves possession of a private key by signing a JWT assertion.
Resource	URI identifying the target protected Resource Server for which a DPP access token is requested.
Service Account	Non-human identity used by an EO external system for machine-to-machine API access.

Chapter 17. Helpdesk Contacts

Need help using the platform or having technical issues? Our Helpdesk team is here to support you.

- Email: EC-HELPDESK-DPP@ec.europa.eu

Responses are provided Monday to Sunday, from 8 AM until 8 PM CET.

- Phone: +32 22960431

Available Monday to Sunday, from 8 AM until 8 PM CET.

> **SECURITY**
> Never include private keys, client assertions or access tokens in a Helpdesk request.